

Restaurant + Franchise + Food Truck Data Model

Logical and physical data model for a growing, data-driven restaurant business

Business scenario

A fast-casual restaurant brand plans to operate corporate restaurants, franchise locations, and mobile food trucks. The owner wants a model that supports day-to-day transactions while also making it easy to analyze sales, customers, menu performance, inventory, suppliers, and unit profitability.

Key design decisions

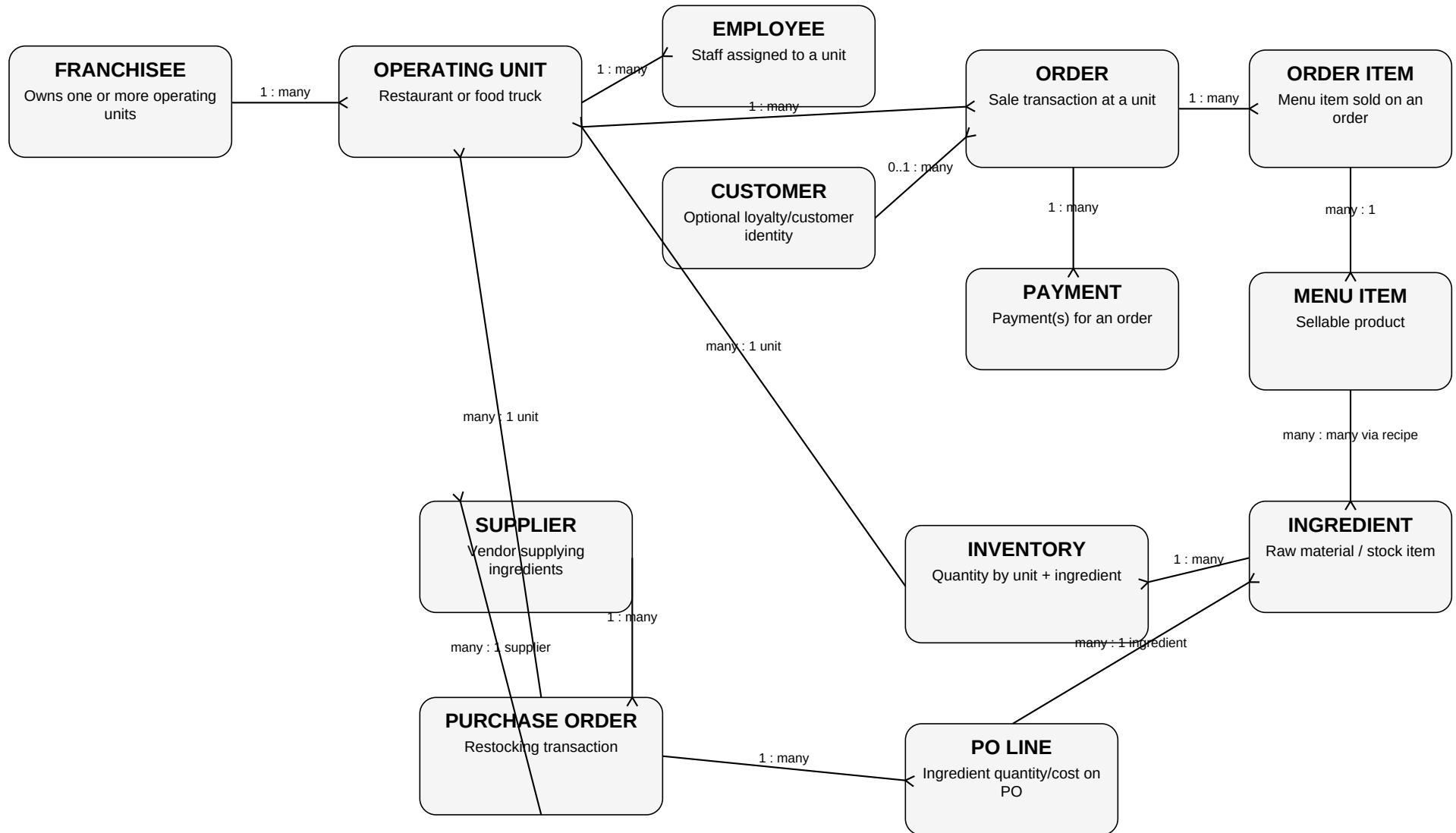
- Model restaurants and food trucks through one Operating Unit entity. `unit_type` identifies RESTAURANT or FOOD_TRUCK, allowing sales, staffing, inventory, and performance logic to be reused.
- Keep sales at order-line grain so menu mix, pricing, discounts, and profitability can be analyzed without losing detail.
- Store ingredient recipes separately from menu items so theoretical food usage and food cost can be compared with actual inventory consumption.
- Separate transactional tables from descriptive/master data. This supports reliable operations today and a future analytics warehouse or star schema.
- Track franchise ownership independently from the operating unit so corporate and franchise-owned locations can be compared consistently.

Primary analytics this model supports

- Revenue and order volume by location, food truck, day, hour, channel, and franchisee.
- Best- and worst-performing menu items, discount impact, average ticket, and product mix.
- Repeat-customer behavior and loyalty trends.
- Ingredient usage, stock-outs, purchasing, supplier cost trends, and estimated food margin.
- Unit-level profitability comparisons between restaurants, franchise locations, and food trucks.

Logical Model

Core business entities and their relationships (technology-agnostic)



Logical relationship highlights

- A Franchisee owns many Operating Units; a corporate-owned unit can use a designated corporate Franchisee record.
- An Operating Unit creates Orders, employs Employees, holds Inventory, and raises Purchase Orders.
- An Order contains many Order Items and may have one or more Payments.
- Menu Items consume Ingredients through a recipe/bridge relationship.

Physical Model - Core Sales & Master Data

Concrete relational design; PK = primary key, FK = foreign key, UQ = unique. Full constraints are in the SQL attachment.

franchisee franchisee_id BIGINT PK franchisee_name VARCHAR(150) contact_email VARCHAR(255) phone VARCHAR(30) agreement_start_date DATE agreement_end_date DATE status VARCHAR(20)	operating_unit unit_id BIGINT PK franchisee_id BIGINT FK unit_type VARCHAR(20) unit_name VARCHAR(150) street_address VARCHAR(200) city VARCHAR(100) state CHAR(2), postal_code VARCHAR(12) latitude/longitude NUMERIC opened_date DATE status VARCHAR(20)	food_truck_detail unit_id BIGINT PK/FK license_plate VARCHAR(30) UQ permit_number VARCHAR(80) commissary_unit_id BIGINT FK vehicle_make VARCHAR(80) vehicle_model VARCHAR(80)	customer customer_id BIGINT PK email VARCHAR(255) UQ phone VARCHAR(30) first_name VARCHAR(80) last_name VARCHAR(80) loyalty_join_date DATE marketing_opt_in BOOLEAN	employee employee_id BIGINT PK home_unit_id BIGINT FK first_name VARCHAR(80) last_name VARCHAR(80) role VARCHAR(50) hire_date DATE hourly_rate NUMERIC(10,2) status VARCHAR(20)
menu_item menu_item_id BIGINT PK sku VARCHAR(50) UQ item_name VARCHAR(150) category VARCHAR(80) base_price NUMERIC(10,2) active_flag BOOLEAN	unit_menu_price unit_id BIGINT PK/FK menu_item_id BIGINT PK/FK effective_from DATE PK effective_to DATE price NUMERIC(10,2) available_flag BOOLEAN	sales_order order_id BIGINT PK unit_id BIGINT FK customer_id BIGINT FK NULL employee_id BIGINT FK NULL order_ts TIMESTAMPTZ channel VARCHAR(20) status VARCHAR(20) subtotal/tax/discount NUMERIC total_amount NUMERIC(12,2)	sales_order_item order_id BIGINT PK/FK line_number SMALLINT PK menu_item_id BIGINT FK quantity NUMERIC(10,2) unit_price NUMERIC(10,2) discount_amount NUMERIC(10,2) line_total NUMERIC(12,2)	payment payment_id BIGINT PK order_id BIGINT FK payment_ts TIMESTAMPTZ payment_method VARCHAR(30) amount NUMERIC(12,2) transaction_reference VARCHAR(100)

Physical Model - Inventory, Recipes & Procurement

Tables supporting purchasing, ingredient consumption, food-cost analysis, and stock management

supplier supplier_id BIGINT PK supplier_name VARCHAR(150) contact_name VARCHAR(120) email VARCHAR(255) phone VARCHAR(30) status VARCHAR(20)	ingredient ingredient_id BIGINT PK ingredient_name VARCHAR(150) uom VARCHAR(20) standard_cost NUMERIC(12,4) active_flag BOOLEAN	menu_item_ingredient menu_item_id BIGINT PK/FK ingredient_id BIGINT PK/FK quantity_required NUMERIC(12,4)
inventory_balance unit_id BIGINT PK/FK ingredient_id BIGINT PK/FK quantity_on_hand NUMERIC(14,4) reorder_point NUMERIC(14,4) updated_at TIMESTAMPTZ	purchase_order po_id BIGINT PK unit_id BIGINT FK supplier_id BIGINT FK po_date DATE expected_date DATE received_date DATE status VARCHAR(20)	purchase_order_line po_id BIGINT PK/FK line_number SMALLINT PK ingredient_id BIGINT FK quantity_ordered NUMERIC(14,4) unit_cost NUMERIC(12,4) quantity_received NUMERIC(14,4)

Important constraints / indexes

- CHECK operating_unit.unit_type IN ('RESTAURANT','FOOD_TRUCK')
- CHECK sales_order.channel IN ('POS','ONLINE','DELIVERY','CATERING')
- UNIQUE customer.email (nullable)
- INDEX sales_order(unit_id, order_ts)
- INDEX sales_order(customer_id, order_ts)
- INDEX sales_order_item(menu_item_id)
- INDEX inventory_balance(ingredient_id)
- INDEX purchase_order(unit_id, po_date)
- Foreign keys enforce referential integrity
- Money stored as NUMERIC, not floating point

Why this physical model scales

- Order and order-item tables are append-heavy and can later be partitioned by date.
- Operating-unit abstraction supports new restaurant or truck locations without schema changes.
- Recipe and ingredient tables enable theoretical food-cost and waste analysis.
- Effective-dated unit pricing preserves historical selling prices.

Analytics Extension & Assessment Comments

How the transactional model can evolve as the company becomes more data-driven

Recommended next step: analytics/star schema

For reporting at larger scale, I would keep the transactional model as the system of record and build a separate analytics layer. A sales fact table at order-line grain could join to date, customer, menu item, operating unit, franchisee, employee, and channel dimensions. Inventory and purchasing would use separate facts because their grains differ.

Example KPIs

- Sales, order count, average ticket, and gross margin by unit / franchisee / food truck.
- Menu item contribution margin, attach rate, discount rate, and daypart performance.
- Customer repeat rate, frequency, recency, and loyalty-member spend.
- Actual vs theoretical ingredient usage, waste proxy, inventory turns, and stock-out frequency.
- Supplier lead time, purchase-price variance, and fill rate.

Comments for the assessment

I modeled restaurants and food trucks as a common operating unit so the same order, employee, inventory, and purchasing structures can support both business types. The logical model focuses on business relationships, while the physical model defines concrete tables, primary/foreign keys, data types, constraints, and indexes. I also separated menu recipes from inventory so the owner can compare what ingredients should have been consumed with what was actually purchased or remains on hand. This design supports current transactional needs and can later feed a star-schema analytics layer without redesigning the core system.

Deliverables

- This PDF: logical model, physical model, design rationale, and analytics extension
- SQL DDL file: concrete table definitions and constraints